

PARQUEO · LOGICIEL ITSM AUTO-HÉBERGÉ

Documentation technique

Architecture, modèle de données, API, moteur de workflows et limites connues.

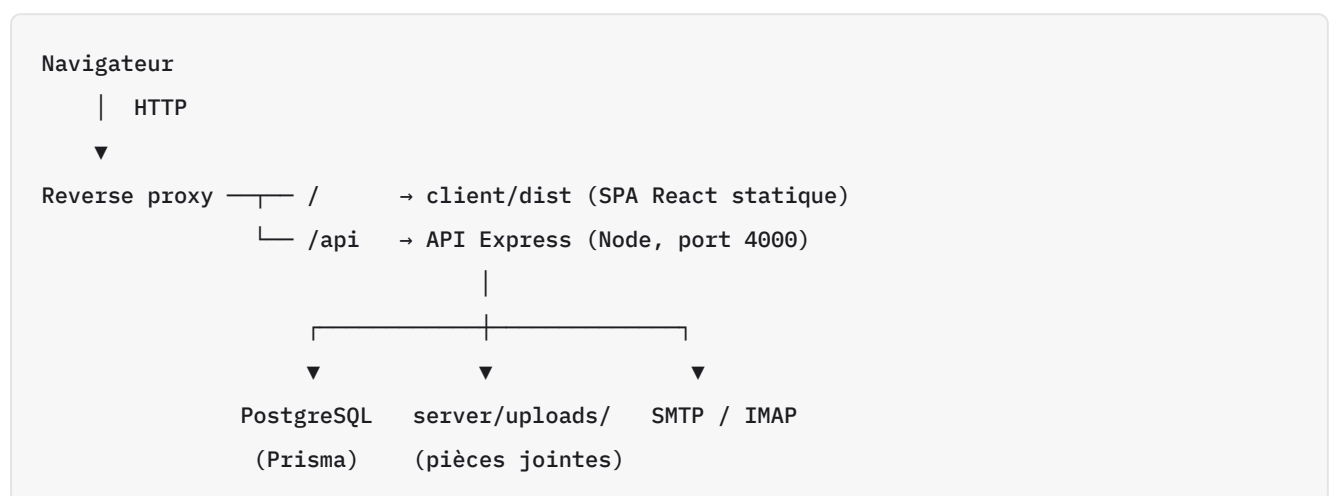
VERSION 0.1.0 30 JUILLET 2026 GITHUB.COM/LCANET2/PARQUEO

Architecture, modèle de données, API et sécurité. Public visé : l'équipe technique qui intègre, exploite ou audite Parqueo.

- | | |
|----------------------------|--------------------------------|
| 1. Vue d'ensemble | 6. Workflows |
| 2. Pile technique | 7. Automatismes |
| 3. Modèle de données | 8. Paramètres |
| 4. Authentification et SSO | 9. Sécurité |
| 5. API HTTP | 10. Dimensionnement et limites |

1. Vue d'ensemble

Parqueo est un logiciel ITSM auto-hébergé : ticketing, inventaire de parc, catalogue de demandes, base de connaissances et automatisation par workflows visuels. Il se déploie en deux processus servis sur une même origine :



L'API est purement JSON (une seule page HTML : l'enquête de satisfaction). Le client appelle `/api` en relatif, ce qui impose de servir front et API sur la **même origine** (voir la documentation d'installation).

2. Pile technique

Couche	Choix	Version
Runtime	Node.js (ESM)	22.x
API	Express	^4.21
ORM	Prisma Client	^5.20
Base	PostgreSQL	14+
Auth	jsonwebtoken (HS256) + bcrypt	^9.0 / ^6.0
Sécurité HTTP	helmet, cors, express-rate-limit	—
Email	nodemailer (sortant), imapflow + mailparser (entrant)	—
Inventaire SNMP	net-snmp	^3.26
Front	React + React Router, build Vite, styles Tailwind	^19 / ^7

Le SSO Microsoft Entra ID est implémenté nativement (`fetch` + `crypto`), sans client OIDC tiers.

3. Modèle de données

PostgreSQL, nommage `snake_case` en base et `camelCase` via Prisma.

3.1 Énumérations

Enum	Valeurs
<code>Role</code>	<code>admin</code> , <code>technician</code> , <code>user</code>
<code>TicketStatus</code>	<code>new</code> , <code>in_progress</code> , <code>waiting</code> , <code>resolved</code> , <code>closed</code>
<code>TicketPriority</code>	<code>low</code> , <code>medium</code> , <code>high</code>
<code>CommentType</code>	<code>comment</code> (message humain), <code>event</code> (journal)
<code>AssetType</code>	<code>pc</code> , <code>printer</code> , <code>server</code> , <code>software</code>
<code>AssetStatus</code>	<code>in_service</code> , <code>in_repair</code> , <code>retired</code>
<code>FieldType</code>	<code>text</code> , <code>textarea</code> , <code>select</code> , <code>date</code> , <code>checkbox</code>

Le tri par statut ou priorité suit l'ordre de l'enum (`new` → `closed` , `low` → `high`), pas l'ordre alphabétique.

3.2 Entités

- **User** — `email` unique, `passwordHash` nullable (les comptes SSO n'en ont pas), `role`, équipe optionnelle.
- **Ticket** — cœur du modèle : titre, description, statut, priorité, et les liens catégorie (requis), auteur (requis), assigné, équipe, actif, formulaire. Porte l'enquête de satisfaction (1/0/null).
- **TicketComment** — messages et journal d'événements, discriminés par `type`. Cascade avec le ticket.
- **Attachment** — métadonnées ; le fichier vit sur le disque.
- **Asset** — inventaire : nom, type, emplacement, date d'achat, état, utilisateur assigné. Champs d'inventaire automatique : `uuid` / `serial` (uniques, clés de déduplication), `source` (`manual` | `agent` | `intune` | `scan`), `lastSeenAt`, caractéristiques matérielles (`manufacturer`, `model`, `os`, `cpu`, `ramMb`, `diskGb`) et `raw` (rapport brut).
- **Software** / **SoftwareInstall** — catalogue de logiciels partagé (nom + éditeur, unique) et installations par actif (avec version).
- **Form** / **FormField** — catalogue de demandes.
- **KbArticle** — base de connaissances.
- **Setting** — table clé/valeur JSON (paramètres globaux, layouts de tableau de bord).
- **Workflow** / **WorkflowStep** / **WorkflowRun** — voir §6.

3.3 Diagramme relationnel

```

Team —< User —< Ticket >— Category
      |   |       |
      |   |       |—< Form —< FormField
      |   |       |—< KbArticle
      |   |—< TicketComment
      |   |—< Attachment
      |   |—< WorkflowRun >— Workflow —< WorkflowStep
      |— Asset —< SoftwareInstall >— Software

```

4. Authentification et SSO

4.1 Jeton

`POST /api/auth/login` vérifie l'email/mot de passe (bcrypt) et émet un JWT **HS256** signé avec `JWT_SECRET`, portant `sub`, `role` et `teamId`. Durée de vie **7 jours**, sans révocation. Le client le stocke et l'envoie en `Authorization: Bearer ...`.

Conséquence : un changement de rôle ou d'équipe, ou une suppression de compte, ne prend effet qu'à l'expiration du jeton (voir §9).

4.2 Visibilité des tickets

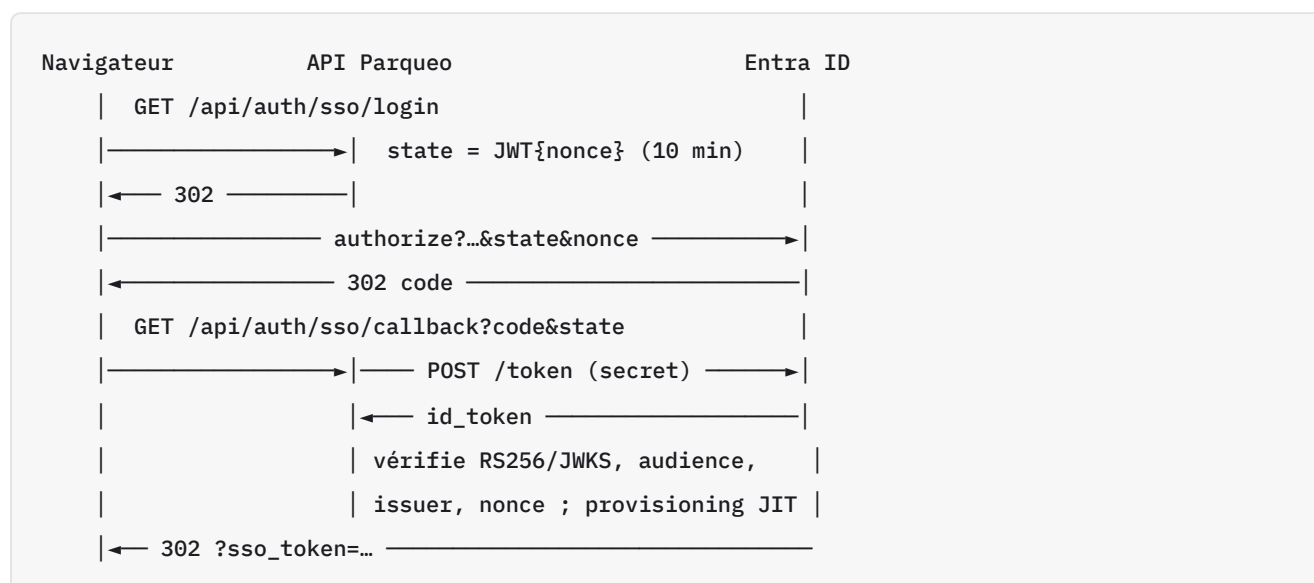
Appliquée dans la requête SQL, à toute lecture :

Rôle	Tickets visibles
admin	tous
technician	ceux qui lui sont assignés, ou non assignés, ou portés par son équipe
user	ceux dont il est l'auteur

Pour l'inventaire, un `user` ne voit que ses actifs assignés, et l'accès peut lui être coupé (paramètre `assetsVisibleToUsers`).

4.3 SSO Microsoft Entra ID

OIDC *authorization code flow*, client confidentiel. Activé dès que `SSO_TENANT_ID`, `SSO_CLIENT_ID` et `SSO_CLIENT_SECRET` sont renseignés (configuration : documentation d'installation §4).



- **Sans session serveur** : le `state` est un JWT signé qui porte le nonce anti-rejeu.
- Le jeton d'identité est vérifié en RS256 (clé publique via JWKS), avec contrôle de l'audience, de l'issuer et du nonce.
- **Provisioning JIT** : un compte inexistant est créé au premier login en rôle `user` ; toute élévation reste un geste d'administration.
- `SSO_ALLOWED_DOMAINS` restreint les domaines email acceptés.
- Les comptes locaux continuent de fonctionner en parallèle (accès de secours).

4.4 Limitation de débit

Seul `/api/auth/login` est limité (20 tentatives / 15 min, les connexions réussies ne comptent pas, pour ne pas bloquer une équipe derrière un même NAT).

5. API HTTP

Préfixe commun `/api`. Toutes les routes exigent un jeton **sauf** `GET /api/health`, `GET /api/auth/config`, `POST /api/auth/login`, les routes SSO et `GET /api/tickets/satisfaction`.

Erreurs : `{ "error": "message" }` avec un statut cohérent (400 validation, 401 auth, 403 rôle, 404 introuvable **ou invisible**, 409 conflit). Le 404 sert aussi l'invisible : on ne distingue pas un ticket inexistant d'un ticket non autorisé.

5.1 Authentification

Méthode	Chemin	Accès
GET	<code>/api/health</code>	public — sonde de vie
GET	<code>/api/auth/config</code>	public — <code>{ sso, intune, snmp }</code> pour l'affichage conditionnel de l'UI
POST	<code>/api/auth/login</code>	public — <code>{ email, password }</code> → <code>{ token, user }</code>
GET	<code>/api/auth/sso/login</code> , <code>/api/auth/sso/callback</code>	public — flux SSO
GET	<code>/api/auth/me</code>	authentifié — profil courant

5.2 Tickets

Méthode	Chemin	Rôle
GET	<code>/api/tickets</code>	tous — filtres + pagination
GET	<code>/api/tickets/:id</code>	tous — détail + commentaires + pièces jointes + état des workflows
POST	<code>/api/tickets</code>	tous — crée, déclenche workflows et notification
PATCH	<code>/api/tickets/:id</code>	admin, technicien
POST	<code>/api/tickets/:id/comments</code>	tous (si visible)
POST/GET	<code>/api/tickets/:id/attachments[/:aId]</code>	tous (si visible) — upload / téléchargement
GET	<code>/api/tickets/satisfaction</code>	public (lien signé) — <code>?token=&value=up down</code>

Filtres : `status`, `priority`, `assigneeId` (id ou `none`), `categoryId`, `teamId`, `q` (titre/description), `sort`. **Pagination** : avec `page`, la réponse est `{ items, total, counts }` (`counts` = nombre de tickets par statut, calculé hors filtre de statut) ; `pageSize` accepte un entier (≤ 500) ou `all`.

5.3 Inventaire

Méthode	Chemin	Rôle
GET	/api/assets	tous (filtré) — ?type=&status=&q=
GET	/api/assets/:id	tous (filtré) — tickets + logiciels installés
POST / PATCH	/api/assets[:id]	admin, technicien
DELETE	/api/assets/:id	admin — 409 si des tickets le référencent
GET	/api/software, /api/software/:id	admin, technicien — catalogue + postes

Ingestion automatique (hors JWT, appelée par des agents/scripts, protégée par le token

INVENTORY_TOKEN — 404 tant qu'il n'est pas défini) :

Méthode	Chemin	Auth
POST	/api/inventory	token — rapport au format normalisé
POST	/api/inventory/glpi	token — rapport natif de l'agent GLPI (zlib/gzip acceptés)
POST	/api/inventory/intune/sync	JWT admin — synchro Microsoft Intune
POST	/api/inventory/snmp/scan	JWT admin — scan réseau SNMP

Mise en œuvre côté exploitant : documentation d'installation §6 ; côté usage : documentation fonctionnelle §9.

5.4 Utilisateurs

Méthode	Chemin	Rôle
GET	/api/users/assignable	admin, technicien
GET / POST / PATCH / DELETE	/api/users[:id]	admin
POST	/api/users/import	admin — import en masse (≤ 1000 lignes)

L'import est **ligne à ligne** (compte rendu created / skipped / error par ligne) ; un mot de passe aléatoire est généré et renvoyé en clair quand la ligne n'en fournit pas. Garde-fous : on ne peut ni retirer son propre rôle admin, ni se supprimer, ni supprimer un compte porteur de tickets (409).

5.5 Workflows, formulaires, base de connaissances, référentiels

Méthode	Chemin	Rôle
GET/POST/PUT/DELETE	/api/workflows[:id]	admin
GET	/api/forms[:id]	tous (actifs ; ?all=1 admin)
POST	/api/forms/:id/submit	tous — crée le ticket

Méthode	Chemin	Rôle
POST/PATCH/DELETE	<code>/api/forms[:id]</code>	admin
GET	<code>/api/kb[:id]</code>	tous (les <code>user</code> ne voient que le publié)
POST/PATCH/DELETE	<code>/api/kb[:id]</code>	admin, techniciens si autorisé
GET	<code>/api/categories</code> , <code>/api/teams</code>	tous
POST	<code>/api/categories</code> , <code>/api/teams</code>	admin

5.6 Paramètres

Méthode	Chemin	Rôle
GET	<code>/api/settings/app</code>	tous (le client adapte l'UI)
PATCH	<code>/api/settings/app</code>	admin
GET/PUT/DELETE	<code>/api/settings/dashboard[...]</code>	selon rôle et permission

6. Workflows

Un workflow est un **graphe orienté** qui automatise le traitement des tickets.

- **Déclencheur** : `ticket_created` OU `status_changed` .
- **Conditions d'entrée** (toutes vraies) : catégorie, formulaire, priorité, statut cible.
- **Blocs** reliés par des fils, avec deux sorties oui/non pour les conditions :

Catégorie	Bloc	Effet
Action	<code>assign_team</code> , <code>assign_user</code>	affecte l'équipe / assigne
Action	<code>set_priority</code> , <code>set_status</code>	change la priorité / le statut
Action	<code>add_note</code> , <code>send_email</code>	journalise / notifie (gabarits <code>{{clé}}</code>)
Action	<code>webhook</code>	POST JSON vers un outil tiers
Branche	<code>condition</code>	teste priorité, statut, catégorie, formulaire ou assignation → oui/non
Attente	<code>wait_assigned</code> , <code>wait_status</code>	parque le ticket jusqu'à la prise en charge / un statut

Parking et reprise. Un ticket qui atteint un bloc d'attente y reste « parqué » ; il repart de ce bloc dès qu'un changement de statut ou d'assignation satisfait la condition. Un même ticket n'entre qu'une fois dans un workflow donné, et une modification faite par un workflow ne peut pas en redéclencher un autre (anti-boucle par construction).

Gabarits. Les notes et emails acceptent des variables `{{ticket.title}}`, `{{author.name}}`, `{{assignee.email}}`, `{{category.name}}`, etc.

Webhook. `POST` `JSON` `{ event, workflow, ticket }`, en-tête optionnel `X-Parqueo-Token`, timeout 5 s ; un échec ne bloque pas le parcours.

7. Automatismes

Quatre traitements tournent dans le processus de l'API, chacun activé seulement si sa configuration est présente.

Notifications email (`mailer.js`), chacune débrayable dans les paramètres :

Notification	Destinataires
Création de ticket	demandeur + assigné
Changement de statut	demandeur + assigné (+ liens de satisfaction à la résolution)
Assignation	le technicien qui reçoit (pas soi-même)
Nouveau message	l'autre partie, jamais l'auteur

Sans `SMTP_HOST`, rien n'est envoyé (les messages sont journalisés). Un échec d'envoi ne fait jamais échouer la requête en cours.

Collecteur email. Si `IMAP_HOST` est configuré : un email d'un utilisateur connu crée un ticket, une réponse dont le sujet contient `Ticket #n` devient un commentaire. Les expéditeurs inconnus et les adresses système sont ignorés.

Clôture automatique. Chaque heure, les tickets `resolved` sans activité depuis `autoCloseDays` jours passent en `closed` (0 = désactivé).

Inventaire automatique. Toutes les sources (script/agent, Intune, SNMP) convergent vers un traitement unique : déduplication par `uuid` puis numéro de série, création ou rafraîchissement de l'actif. Les champs **remontés** (matériel, logiciels, dernière remontée) sont rafraîchis à chaque rapport ; les champs **gérés par l'humain** (nom, type, emplacement, état, utilisateur assigné) ne sont jamais écrasés. Le signalement des actifs « périmés » est purement visuel : aucun statut n'est modifié automatiquement.

8. Paramètres

Configuration globale (table `settings`, clé `app.config`), éditée dans la page Paramètres (admin) et lue par tous les comptes pour adapter l'interface.

Clé	Défaut	Effet
<code>ticketDefaultPriority</code>	<code>medium</code>	priorité par défaut
<code>userCanSetPriority</code>	<code>true</code>	un utilisateur peut choisir la priorité
<code>autoCloseDays</code>	<code>7</code>	délai de clôture automatique (0 = désactivé)
<code>satisfactionSurvey</code>	<code>true</code>	enquête un clic à la résolution
<code>notifyOnCreate/Status/Assign/Comment</code>	<code>true</code>	notifications email
<code>kbSuggest</code>	<code>true</code>	suggestions d'articles à la création
<code>kbTechniciansWrite</code>	<code>true</code>	rédaction de la base de connaissances par les techniciens
<code>assetsVisibleToUsers</code>	<code>true</code>	inventaire visible des utilisateurs
<code>assetStaleDays</code>	<code>30</code>	délai au-delà duquel un actif automatique est signalé périmé (0 = jamais)
<code>dashboardPersonalTechnician/User</code>	<code>false</code>	personnalisation du tableau de bord

Écriture réservée aux admins, avec validation stricte : une clé inconnue ou une valeur invalide est refusée (400).

9. Sécurité

En place :

- `helmet` (en-têtes HTTP) et `cors` restreint à `CLIENT_ORIGIN` ;
- mots de passe bcrypt (coût 10), 8 caractères minimum ;
- limitation de débit sur le login ;
- autorisation à deux niveaux : rôle, puis visibilité appliquée **dans la requête SQL** ;
- pas de cookie de session → **pas de surface CSRF** ;
- SSO vérifiant signature, audience, issuer et nonce ;
- pièces jointes jamais servies en statique (téléchargement authentifié, avec revérification de la visibilité du ticket).

À connaître avant mise en production :

1. **Jeton en `localStorage`** : une faille XSS dans le front exposerait le jeton. Le choix évite CSRF et simplifie le déploiement.
2. **Jeton non révoquant (7 j)** : un changement de rôle ou une suppression de compte ne coupe pas les sessions en cours ; pour une révocation immédiate, il faut faire tourner `JWT_SECRET` (déconnecte tout le monde).

3. `sso_token` dans l'URL de retour : retiré immédiatement de la barre d'adresse, mais susceptible d'apparaître dans les journaux du reverse proxy — filtrez les query strings.
 4. **Collecteur email** : l'expéditeur est vérifié par simple correspondance d'adresse (ni SPF/DKIM/DMARC) ; une adresse usurpée peut créer un ticket. À filtrer en amont sur le serveur de messagerie.
 5. **Pièces jointes** : liste blanche d'extensions, sans analyse du contenu.
 6. **Débit** : seul le login est limité ; ajoutez un quota au reverse proxy si l'API est exposée publiquement.
 7. **Webhook sortant** : un administrateur peut cibler une IP interne (SSRF) — le rôle admin est de toute façon de confiance.
 8. **Compte de seed par défaut** (`admin@parqueo.local` / `admin1234`) : à changer impérativement au premier démarrage.
-

10. Dimensionnement et limites

- L'API est un processus Node mono-thread léger : quelques dizaines d'utilisateurs simultanés tiennent sur 1 vCPU / 1 Go, PostgreSQL compris.
- **Pagination serveur** uniquement sur les tickets ; l'inventaire, les utilisateurs et la base de connaissances sont paginés côté client. Le tableau de bord calcule ses métriques à partir de tous les tickets visibles — à plusieurs dizaines de milliers de tickets, prévoir des agrégats SQL.
- **Pièces jointes** : stockées sur disque, **jamais purgées** (y compris à la suppression d'un ticket). Surveiller l'espace de `server/uploads/`.
- **Journal d'audit** : la traçabilité repose sur les événements de ticket ; il n'existe pas de journal distinct des créations de comptes, changements de rôle ou de paramètres.